

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Пензенский государственный университет»
(ПГУ)
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе № 3

по дисциплине «Логика и основы алгоритмизации в инженерных задачах»

**Тема: «ДИНАМИЧЕСКИЕ СТРУКТУРЫ ДАННЫХ (СПИСКИ, СТЕК,
ОЧЕРЕДЬ, ПРИОРИТЕТНАЯ ОЧЕРЕДЬ)»**

Выполнили:

студенты гр. 25BBB1:

Ячменев М. А.

Аверин А. А.

Приняли:

к.т.н., доцент Юрова О. В.

ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Изучение принципов организации и функционирования динамических структур данных (односвязные линейные списки, стек, очередь, приоритетная очередь). Приобретение практических навыков динамического управления оперативной памятью в языке программирования Си с использованием функций `malloc()` и `free()`, разработки алгоритмов добавления, извлечения, сквозного просмотра, поиска и удаления элементов в связанных структурах данных.

2. Лабораторное задание

В соответствии с методическими указаниями к лабораторной работе № 3 требуется выполнить следующие задания:

- 1. Изучить базовую реализацию односвязного линейного списка, содержащего текстовые наименования объектов (`inf[256]`), функции добавления (`spstore`), просмотра (`review`), поиска (`find`) и удаления (`del`).
- 2. На основе базовой связной структуры реализовать структуру данных Очередь (`Queue`), работающую по принципу FIFO (First In, First Out — первым пришел, первым обслужен), с операциями постановки в хвост и извлечения из головы.
- 3. На основе приведенного связного списка реализовать структуру данных Стек (`Stack`), функционирующую по принципу LIFO (Last In, First Out — последним пришел, первым обслужен), с операциями помещения в вершину (`push`) и извлечения из вершины (`pop`).
- 4. Реализовать структуру данных Приоритетная очередь (`Priority Queue`) путем упорядоченного включения элементов в список в соответствии с числовым приоритетом объекта (элемент с более высоким приоритетом размещается перед элементом с меньшим приоритетом, а при равных приоритетах сохраняется естественный порядок поступления FIFO).
- 5. Реализовать функции поиска и выборочного удаления элементов по наименованию в приоритетной очереди, а также функции полной очистки выделенной динамической памяти.
- 6. Разработать демонстрационный модуль и интерактивное меню пользователя в среде Microsoft Visual Studio на языке Си, провести тестирование всех реализованных структур и составить подробный отчет о результатах работы.

3. Теоретические сведения

3.1. Особенности динамических структур данных и управление памятью

В отличие от статических структур данных (таких как стандартные одномерные и многомерные массивы), размер которых фиксируется на этапе компиляции и не может изменяться в процессе выполнения программы, динамические структуры данных обладают следующими ключевыми свойствами:

- Динамическое выделение памяти: память под каждый элемент (узел) выделяется из свободной области оперативной памяти (кучи, heap) непосредственно в момент создания элемента по мере необходимости.
- Связность через указатели: узлы структуры не обязаны располагаться в смежных ячейках оперативной памяти; логическая связь между элементами обеспечивается явным хранением адреса следующего узла в поле-указателе.
- Отсутствие жесткого ограничения на размер: структура может неограниченно расти вплоть до полного исчерпания физической оперативной памяти вычислительной системы и легко уменьшаться при освобождении узлов.

В языке Си управление динамической памятью осуществляется посредством стандартных функций библиотеки `<stdlib.h>`:

- `malloc(size)` — выделяет непрерывный блок памяти размером `size` байт и возвращает нетипизированный указатель (`void*`) на его начало, либо `NULL` при нехватке памяти.
- `free(ptr)` — освобождает ранее выделенный блок памяти по адресу `ptr`, возвращая его операционной системе для повторного использования.

Базовым строительным блоком динамического списка является узел (структура `node`), определяемый следующим образом:

```
struct node {  
    char inf[256];           // полезная информация (наименование объекта)  
    int priority;            // приоритет элемента (для приоритетной очереди)  
    struct node *next;       // указатель на следующий элемент списка  
};
```

3.2. Принцип функционирования Стекa (LIFO)

Стек (Stack) — динамическая структура данных, работающая по принципу LIFO (Last In, First Out — «последним пришел, первым обслужен»). Доступ к

элементам осуществляется исключительно через один конец структуры, называемый вершиной стека (TOP):

- Операция push (проталкивание): новый элемент связывается с текущей вершиной, после чего указатель вершины смещается на вновь добавленный узел. Сложность операции — $O(1)$.
- Операция pop (выталкивание): из вершины извлекается информационное поле, указатель вершины переносится на следующий элемент (next), а память прежней вершины освобождается вызовом free(). Сложность операции — $O(1)$.

3.3. Принцип функционирования Очереди (FIFO)

Очередь (Queue) — структура данных, организованная по дисциплине обслуживания FIFO (First In, First Out — «первым пришел, первым ушел»). Для эффективного функционирования используются два независимых указателя:

- Указатель на начало (HEAD): через него осуществляется извлечение обслуженных элементов (dequeue).
- Указатель на конец (TAIL): через него осуществляется добавление новых элементов в хвост очереди (enqueue).

Использование двух указателей обеспечивает константную временную сложность $O(1)$ как для операции добавления в хвост, так и для извлечения из головы.

3.4. Принцип функционирования Приоритетной очереди

Приоритетная очередь (Priority Queue) — структура данных, в которой каждый элемент помимо данных несет характеристику важности (приоритет priority). Обслуживание элементов производится строго в порядке убывания их приоритета (наивысший приоритет обслуживается первым):

- Алгоритм упорядоченной вставки (enqueue): при поступлении нового элемента список просматривается от головы; элемент размещается перед первым встреченным узлом, имеющим строго меньший приоритет.
- Стабильность очереди (FIFO для равных приоритетов): если в очереди уже присутствуют элементы с таким же приоритетом, новый элемент помещается за ними (условие $\text{next} \rightarrow \text{priority} \geq \text{priority}$), что гарантирует справедливую очередность обслуживания.
- Извлечение максимального элемента (dequeue): поскольку список постоянно поддерживается в отсортированном по невозрастанию приоритетов

состоянии, элемент с наивысшим приоритетом всегда находится в голове очереди (HEAD), что позволяет извлечь его за время $O(1)$.

1. Стек (Stack — LIFO: Last In, First Out)



2. Очередь (Queue — FIFO: First In, First Out)



3. Приоритетная очередь (Priority Queue — упорядочение по приоритету)



* При одинаковом значении приоритета новый элемент ставится ПОСЛЕ существующего (сохранение FIFO)

Рисунок 1 – Схемы организации структур данных: Стек (LIFO), Очередь (FIFO) и Приоритетная очередь

4. Описание программной реализации

Программный комплекс разработан в среде Microsoft Visual Studio на стандартном языке Си (C99/C11). Проект структурирован в виде консольного приложения и включает следующие компоненты:

- alg3lr.sln / alg3lr.slnx — файлы решений Visual Studio для удобного открытия проекта в один клик в среде разработки.
- alg3lr.vcxproj — файл конфигурации проекта C/C++ с параметрами сборки под платформы x86 и x64.
- alg3lr.c — основной файл исходного кода, содержащий полную реализацию структур данных, демонстрационного тестирования и пользовательского диалогового меню.

Программа состоит из следующих функциональных блоков:

- Модуль стека: функции `stack_push()` — выделение узла и включение в вершину `stack_top`; `stack_pop()` — удаление вершины с возвратом данных; `stack_review()` — итеративный вывод элементов; `stack_clear()` — полное циклическое освобождение памяти.
- Модуль очереди: функции `queue_enqueue()` — добавление в `queue_tail` с обновлением связей; `queue_dequeue()` — извлечение из `queue_head`; `queue_review()` — просмотр от головы к хвосту; `queue_clear()` — очистка всех узлов очереди.
- Модуль приоритетной очереди: функции `pq_enqueue()` — однократный линейный поиск позиции вставки по убыванию поля `priority`; `pq_dequeue()` — немедленное извлечение элемента из головы списка `pq_head`; `pq_find()` — последовательный поиск элемента по строковому наименованию `strcmp()`; `pq_del()` — перелинковка указателей и освобождение найденного узла; `pq_review()` и `pq_clear()`.
- Диалоговый интерфейс: функция `main()` реализует текстовое консольное меню с поддержкой кодовой страницы Windows-1251 для корректного отображения кириллических символов, обработкой ошибок ввода и пунктом 1 для запуска автоматического демонстрационного теста.

Таблица 1 – Сравнительная характеристика реализованных структур данных

Структура данных	Принцип доступа	Сложность вставки	Сложность извлечения	Точка добавления	Точка извлечения
Стек (Stack)	LIFO	O(1)	O(1)	Вершина (TOP)	Вершина (TOP)
Очередь (Queue)	FIFO	O(1)	O(1)	Хвост (TAIL)	Голова (HEAD)
Приоритетная очередь	По приоритету (Priority)	O(N)	O(1)	По позиции приоритета	Голова (HEAD)
Линейный список	Произвольный	O(1) в конец	O(N) по ключу	Конец (last)	По значению (del)

5. Листинг программы на языке Си

Ниже приведен полный текст исходного кода разработанной программы (файл `alg3lr.c`), реализующей все требования лабораторной работы № 3:

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```

#include <locale.h>
#include <windows.h>

/*
 * =====
 * Пензенский государственный университет
 * Кафедра «Вычислительная техника»
 * Дисциплина: «Логика и основы алгоритмизации в инженерных задачах»
 *
 * ЛАБОРАТОРНАЯ РАБОТА № 3
 * Тема: «Динамические структуры данных (Списки, Стек, Очередь, Приоритетная очередь)»
 *
 * Выполнили: студенты группы 25BVB1
 *             Ячменев М. А., Аверин А. А.
 * Приняли:    К.Т.Н., доцент Юрова О. В.
 *             ст. преподаватель Деев М. В.
 *
 * Пенза, 2026 г.
 * =====
 */

/* Структура элемента динамического списка */
struct node {
    char inf[256];          // полезная информация (наименование объекта)
    int priority;           // приоритет элемента (для приоритетной очереди)
    struct node *next;      // ссылка на следующий элемент списка
};

/* =====
 * РАЗДЕЛ 1. БАЗОВЫЙ ОДНОСВЯЗНЫЙ СПИСОК (по методическому пособию)
 * ===== */
struct node *head = NULL;
struct node *last = NULL;

/* Функция создания нового узла списка */
struct node *get_struct(const char *s) {
    struct node *p = (struct node*)malloc(sizeof(struct node));
    if (p == NULL) {
        printf("Ошибка выделения динамической памяти!\n");
        exit(1);
    }
    strncpy(p->inf, s, 255);
    p->inf[255] = '\0';
    p->priority = 0;
    p->next = NULL;
    return p;
}

/* Последовательное добавление элемента в конец списка */
void spstore(const char *s) {
    struct node *p = get_struct(s);
    if (head == NULL) {
        head = p;
        last = p;
    } else {
        last->next = p;
        last = p;
    }
    printf("[Список] Элемент \"%s\" добавлен в список.\n", s);
}

/* Просмотр содержимого односвязного списка */
void review(void) {
    struct node *struc = head;
    if (struc == NULL) {
        printf("[Список] Список пуст.\n");
        return;
    }
}

```

```

printf("[Список] Содержимое списка:\n");
int i = 1;
while (struc != NULL) {
    printf("  %d) Имя: \"%s\"\n", i++, struc->inf);
    struc = struc->next;
}
}

/* Поиск элемента по содержимому */
struct node *find(const char *name) {
    struct node *struc = head;
    int pos = 1;
    while (struc != NULL) {
        if (strcmp(name, struc->inf) == 0) {
            printf("[Список] Элемент \"%s\" найден на позиции %d.\n", name, pos);
            return struc;
        }
        struc = struc->next;
        pos++;
    }
    printf("[Список] Элемент \"%s\" не найден!\n", name);
    return NULL;
}

/* Удаление элемента по содержимому */
void del(const char *name) {
    struct node *struc = head;
    struct node *prev = NULL;
    int flag = 0;

    if (head == NULL) {
        printf("[Список] Список пуст, удаление невозможно.\n");
        return;
    }

    // Если удаляемый элемент - первый
    if (strcmp(name, struc->inf) == 0) {
        flag = 1;
        head = struc->next;
        if (head == NULL) last = NULL;
        free(struc);
        printf("[Список] Элемент \"%s\" (голова списка) успешно удален.\n", name);
        return;
    }

    prev = struc;
    struc = struc->next;
    while (struc != NULL) {
        if (strcmp(name, struc->inf) == 0) {
            flag = 1;
            prev->next = struc->next;
            if (struc == last) {
                last = prev;
            }
            free(struc);
            printf("[Список] Элемент \"%s\" успешно удален.\n", name);
            return;
        }
        prev = struc;
        struc = struc->next;
    }

    if (flag == 0) {
        printf("[Список] Элемент \"%s\" для удаления не найден.\n", name);
    }
}

/* Очистка списка */

```

```

void list_clear(void) {
    struct node *curr = head;
    while (curr != NULL) {
        struct node *tmp = curr;
        curr = curr->next;
        free(tmp);
    }
    head = NULL;
    last = NULL;
}

/* =====
 * РАЗДЕЛ 2. РЕАЛИЗАЦИЯ СТЕКА (STACK - LIFO: Last In, First Out)
 * Добавление в вершину (push), извлечение из вершины (pop).
 * ===== */
struct node *stack_top = NULL;

/* Добавление элемента в стек */
void stack_push(const char *name) {
    struct node *p = (struct node*)malloc(sizeof(struct node));
    if (p == NULL) {
        printf("[Стек] Ошибка выделения памяти!\n");
        return;
    }
    strncpy(p->inf, name, 255);
    p->inf[255] = '\0';
    p->priority = 0;
    p->next = stack_top; // Новый узел указывает на прежнюю вершину
    stack_top = p;      // Новой вершиной становится созданный узел
    printf("[Стек] Помещен элемент (push): \"%s\".\n", name);
}

/* Извлечение элемента из стека */
int stack_pop(char *out_name) {
    if (stack_top == NULL) {
        printf("[Стек] Ошибка: стек пуст!\n");
        return 0;
    }
    struct node *temp = stack_top;
    if (out_name != NULL) {
        strcpy(out_name, temp->inf);
    }
    printf("[Стек] Извлечен верхний элемент (pop): \"%s\".\n", temp->inf);
    stack_top = stack_top->next; // Смещаем вершину на следующий элемент
    free(temp);                 // Освобождаем память удаленного элемента
    return 1;
}

/* Просмотр содержимого стека */
void stack_review(void) {
    struct node *curr = stack_top;
    if (curr == NULL) {
        printf("[Стек] Стек пуст.\n");
        return;
    }
    printf("[Стек] Содержимое (от вершины к основанию):\n");
    int i = 1;
    while (curr != NULL) {
        printf(" %d) \"%s\" %s\n", i, curr->inf, (i == 1) ? "<-- Вершина (TOP)" : "");
        curr = curr->next;
        i++;
    }
}

/* Полная очистка стека */
void stack_clear(void) {
    struct node *curr = stack_top;

```

```

    while (curr != NULL) {
        struct node *tmp = curr;
        curr = curr->next;
        free(tmp);
    }
    stack_top = NULL;
}

/* =====
 * РАЗДЕЛ 3. РЕАЛИЗАЦИЯ ОЧЕРЕДИ (QUEUE - FIFO: First In, First Out)
 * Добавление в конец (хвост), извлечение из начала (головы).
 * ===== */
struct node *queue_head = NULL;
struct node *queue_tail = NULL;

/* Добавление элемента в очередь (enqueue) */
void queue_enqueue(const char *name) {
    struct node *p = (struct node*)malloc(sizeof(struct node));
    if (p == NULL) {
        printf("[Очередь] Ошибка выделения памяти!\n");
        return;
    }
    strncpy(p->inf, name, 255);
    p->inf[255] = '\0';
    p->priority = 0;
    p->next = NULL;

    if (queue_head == NULL) {
        // Очередь пуста: голова и хвост ссылаются на новый узел
        queue_head = p;
        queue_tail = p;
    } else {
        // Добавляем узел за текущим хвостом
        queue_tail->next = p;
        queue_tail = p;
    }
    printf("[Очередь] Добавлен в хвост (enqueue): \"%s\".\n", name);
}

/* Извлечение элемента из очереди (dequeue) */
int queue_dequeue(char *out_name) {
    if (queue_head == NULL) {
        printf("[Очередь] Ошибка: очередь пуста!\n");
        return 0;
    }
    struct node *temp = queue_head;
    if (out_name != NULL) {
        strcpy(out_name, temp->inf);
    }
    printf("[Очередь] Обслужен из головы (dequeue): \"%s\".\n", temp->inf);
    queue_head = queue_head->next;
    if (queue_head == NULL) {
        queue_tail = NULL; // Очередь опустела
    }
    free(temp);
    return 1;
}

/* Просмотр содержимого очереди */
void queue_review(void) {
    struct node *curr = queue_head;
    if (curr == NULL) {
        printf("[Очередь] Очередь пуста.\n");
        return;
    }
    printf("[Очередь] Содержимое (от головы к хвосту):\n");
    int i = 1;

```

```

    while (curr != NULL) {
        printf(" %d) \"%s\" %s\n", i, curr->inf,
            (curr == queue_head) ? "<-- Голова (HEAD)" : ((curr == queue_tail) ? "<-- Хвост
(TAIL)" : ""));
        curr = curr->next;
        i++;
    }
}

/* Очистка очереди */
void queue_clear(void) {
    struct node *curr = queue_head;
    while (curr != NULL) {
        struct node *tmp = curr;
        curr = curr->next;
        free(tmp);
    }
    queue_head = NULL;
    queue_tail = NULL;
}

/* =====
* РАЗДЕЛ 4. РЕАЛИЗАЦИЯ ПРИОРИТЕТНОЙ ОЧЕРЕДИ (PRIORITY QUEUE)
* Элемент с большим приоритетом становится ПЕРЕД элементом с меньшим приоритетом.
* При равных приоритетах строго соблюдается очередность FIFO.
* ===== */
struct node *pq_head = NULL;

/* Добавление в приоритетную очередь с упорядочением по приоритету */
void pq_enqueue(const char *name, int priority) {
    struct node *p = (struct node*)malloc(sizeof(struct node));
    if (p == NULL) {
        printf("[Приоритетная очередь] Ошибка выделения памяти!\n");
        return;
    }
    strncpy(p->inf, name, 255);
    p->inf[255] = '\0';
    p->priority = priority;
    p->next = NULL;

    // Случай 1: Список пуст или приоритет нового элемента строго выше головы списка
    if (pq_head == NULL || priority > pq_head->priority) {
        p->next = pq_head;
        pq_head = p;
    } else {
        // Случай 2: Ищем позицию вставки.
        // Проходим узлы, пока приоритет следующего узла больше либо равен нашему (>= гарантирует
FIFO).
        struct node *curr = pq_head;
        while (curr->next != NULL && curr->next->priority >= priority) {
            curr = curr->next;
        }
        p->next = curr->next;
        curr->next = p;
    }
    printf("[Приоритетная очередь] Добавлен: \"%s\" (Приоритет: %d).\n", name, priority);
}

/* Извлечение элемента с наивысшим приоритетом (из головы очереди) */
int pq_dequeue(char *out_name, int *out_priority) {
    if (pq_head == NULL) {
        printf("[Приоритетная очередь] Очередь пуста!\n");
        return 0;
    }
    struct node *temp = pq_head;
    if (out_name != NULL) {
        strcpy(out_name, temp->inf);
    }

```

```

    }
    if (out_priority != NULL) {
        *out_priority = temp->priority;
    }
    printf("[Приоритетная очередь] Извлечен наивысший приоритет: \"%s\" (Приоритет: %d).\n",
        temp->inf, temp->priority);
    pq_head = pq_head->next;
    free(temp);
    return 1;
}

/* Поиск элемента по наименованию */
struct node *pq_find(const char *name) {
    struct node *curr = pq_head;
    int pos = 1;
    while (curr != NULL) {
        if (strcmp(curr->inf, name) == 0) {
            printf("[Приоритетная очередь] Найден элемент \"%s\" на позиции %d (Приоритет: %d).\n",
                name, pos, curr->priority);
            return curr;
        }
        curr = curr->next;
        pos++;
    }
    printf("[Приоритетная очередь] Элемент \"%s\" не найден!\n", name);
    return NULL;
}

/* Удаление элемента по наименованию */
int pq_del(const char *name) {
    if (pq_head == NULL) {
        printf("[Приоритетная очередь] Очередь пуста, удаление невозможно.\n");
        return 0;
    }

    struct node *curr = pq_head;
    struct node *prev = NULL;

    // Если удаляемый узел - голова
    if (strcmp(curr->inf, name) == 0) {
        pq_head = curr->next;
        free(curr);
        printf("[Приоритетная очередь] Элемент \"%s\" успешно удален.\n", name);
        return 1;
    }

    // Поиск узла в глубине очереди
    while (curr != NULL && strcmp(curr->inf, name) != 0) {
        prev = curr;
        curr = curr->next;
    }

    if (curr == NULL) {
        printf("[Приоритетная очередь] Элемент \"%s\" для удаления не найден.\n", name);
        return 0;
    }

    prev->next = curr->next;
    free(curr);
    printf("[Приоритетная очередь] Элемент \"%s\" успешно удален.\n", name);
    return 1;
}

/* Просмотр содержимого приоритетной очереди */
void pq_review(void) {
    struct node *curr = pq_head;
    if (curr == NULL) {
        printf("[Приоритетная очередь] Очередь пуста.\n");
    }
}

```

```

        return;
    }
    printf("[Приоритетная очередь] Содержимое (по убыванию приоритета):\n");
    int i = 1;
    while (curr != NULL) {
        printf("  %d) \"%s\" | Приоритет: %-2d %s\n", i, curr->inf, curr->priority,
            (i == 1) ? "<--- НАИВЫСШИЙ ПРИОРИТЕТ" : "");
        curr = curr->next;
        i++;
    }
}

/* Очистка приоритетной очереди */
void pq_clear(void) {
    struct node *curr = pq_head;
    while (curr != NULL) {
        struct node *tmp = curr;
        curr = curr->next;
        free(tmp);
    }
    pq_head = NULL;
}

/* =====
 * РАЗДЕЛ 5. АВТОМАТИЧЕСКОЕ ДЕМОСТРАЦИОННОЕ ТЕСТИРОВАНИЕ
 * ===== */
void run_demo(void) {
    char buf[256];
    int pr = 0;

    printf("\n===== \n");
    printf("  ДЕМОСТРАЦИОННОЕ ТЕСТИРОВАНИЕ ВСЕХ СТРУКТУР ДАННЫХ      \n");
    printf("===== \n");

    /* 1. Тестирование стека */
    printf("\n--- ТЕСТ 1: СТЕК (STACK - LIFO) --- \n");
    stack_clear();
    stack_push("Документ_1");
    stack_push("Документ_2");
    stack_push("Документ_3");
    stack_review();

    stack_pop(buf);
    stack_review();
    stack_clear();

    /* 2. Тестирование очереди */
    printf("\n--- ТЕСТ 2: ОЧЕРЕДЬ (QUEUE - FIFO) --- \n");
    queue_clear();
    queue_enqueue("Клиент_А");
    queue_enqueue("Клиент_Б");
    queue_enqueue("Клиент_В");
    queue_review();

    queue_dequeue(buf);
    queue_review();
    queue_clear();

    /* 3. Тестирование приоритетной очереди */
    printf("\n--- ТЕСТ 3: ПРИОРИТЕТНАЯ ОЧЕРЕДЬ (PRIORITY QUEUE) --- \n");
    pq_clear();
    pq_enqueue("Фоновая_синхронизация", 1);
    pq_enqueue("Критический_сбой_системы", 10);
    pq_enqueue("Отправка_логов", 2);
    pq_enqueue("Срочный_запрос_пользователя", 8);
    pq_enqueue("Запрос_базы_данных_1", 5);
    pq_enqueue("Запрос_базы_данных_2", 5); // Проверка сохранения очередности FIFO при равных

```

```

приоритетах
pq_review();

printf("\n[Тестирование поиска элемента]:\n");
pq_find("Отправка_логов");
pq_find("Несуществующий_элемент");

printf("\n[Тестирование удаления элемента]:\n");
pq_del("Отправка_логов");
pq_review();

printf("\n[Последовательное обслуживание очереди по приоритету]:\n");
while (pq_dequeue(buf, &pr)) {
    // Извлекаем элементы до полного опустошения
}
pq_review();
pq_clear();

printf("\n===== \n");
printf("                ТЕСТИРОВАНИЕ УСПЕШНО ЗАВЕРШЕНО!                \n");
printf("===== \n\n");
}

/* =====
 * РАЗДЕЛ 6. ГЛАВНОЕ МЕНЮ ПРОГРАММЫ
 * ===== */
int main(void) {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Russian");

    int choice = -1;
    char name[256];
    int priority = 0;

    while (choice != 0) {
        printf("\n===== \n");
        printf("    ЛАБОРАТОРНАЯ РАБОТА №3: ДИНАМИЧЕСКИЕ СТРУКТУРЫ    \n");
        printf("===== \n");
        printf(" 1. Запустить автоматическое демонстрационное тестирование\n");
        printf(" 2. СТЕК (LIFO) -> Добавить элемент (push)\n");
        printf(" 3. СТЕК (LIFO) -> Извлечь элемент (pop)\n");
        printf(" 4. СТЕК (LIFO) -> Просмотреть содержимое\n");
        printf(" 5. ОЧЕРЕДЬ (FIFO) -> Добавить элемент (enqueue)\n");
        printf(" 6. ОЧЕРЕДЬ (FIFO) -> Извлечь элемент (dequeue)\n");
        printf(" 7. ОЧЕРЕДЬ (FIFO) -> Просмотреть содержимое\n");
        printf(" 8. ПРИОРИТЕТНАЯ ОЧЕРЕДЬ -> Добавить с приоритетом\n");
        printf(" 9. ПРИОРИТЕТНАЯ ОЧЕРЕДЬ -> Извлечь элемент с макс. приоритетом\n");
        printf("10. ПРИОРИТЕТНАЯ ОЧЕРЕДЬ -> Поиск по наименованию\n");
        printf("11. ПРИОРИТЕТНАЯ ОЧЕРЕДЬ -> Удаление по наименованию\n");
        printf("12. ПРИОРИТЕТНАЯ ОЧЕРЕДЬ -> Просмотреть все элементы\n");
        printf(" 0. Выход из программы\n");
        printf("----- \n");
        printf("Выберите пункт меню: ");

        if (scanf("%d", &choice) != 1) {
            while (getchar() != '\n');
            printf("Некорректный ввод! Пожалуйста, введите номер пункта меню.\n");
            continue;
        }

        switch (choice) {
            case 1:
                run_demo();
                break;
            case 2:
                printf("Введите наименование объекта для помещения в стек: ");

```

```

        scanf("%s", name);
        stack_push(name);
        break;
    case 3:
        stack_pop(name);
        break;
    case 4:
        stack_review();
        break;
    case 5:
        printf("Введите наименование объекта для добавления в очередь: ");
        scanf("%s", name);
        queue_enqueue(name);
        break;
    case 6:
        queue_dequeue(name);
        break;
    case 7:
        queue_review();
        break;
    case 8:
        printf("Введите наименование объекта: ");
        scanf("%s", name);
        printf("Введите приоритет (целое число, чем больше, тем выше): ");
        if (scanf("%d", &priority) == 1) {
            pq_enqueue(name, priority);
        } else {
            printf("Ошибка: неверный формат приоритета.\n");
            while (getchar() != '\n');
        }
        break;
    case 9:
        pq_dequeue(name, &priority);
        break;
    case 10:
        printf("Введите наименование для поиска: ");
        scanf("%s", name);
        pq_find(name);
        break;
    case 11:
        printf("Введите наименование для удаления: ");
        scanf("%s", name);
        pq_del(name);
        break;
    case 12:
        pq_review();
        break;
    case 0:
        printf("Очистка выделенной памяти и завершение программы...\n");
        list_clear();
        stack_clear();
        queue_clear();
        pq_clear();
        printf("Память успешно освобождена. До свидания!\n");
        break;
    default:
        printf("Неверный пункт меню! Введите число от 0 до 12.\n");
        break;
}

return 0;
}

```

6. Результаты работы программы

Для подтверждения корректности функционирования всех реализованных алгоритмов было проведено комплексное тестирование в автоматическом демонстрационном режиме (пункт меню 1). Ниже приведен фактический протокол вывода в консоль:

```
=====
      ДЕМОСТРАЦИОННОЕ ТЕСТИРОВАНИЕ ВСЕХ СТРУКТУР ДАННЫХ
=====

--- ТЕСТ 1: СТЕК (STACK - LIFO) ---
[Стек] Помещен элемент (push): "Документ_1".
[Стек] Помещен элемент (push): "Документ_2".
[Стек] Помещен элемент (push): "Документ_3".
[Стек] Содержимое (от вершины к основанию):
  1) "Документ_3" <-- Вершина (TOP)
  2) "Документ_2"
  3) "Документ_1"
[Стек] Извлечен верхний элемент (pop): "Документ_3".
[Стек] Содержимое (от вершины к основанию):
  1) "Документ_2" <-- Вершина (TOP)
  2) "Документ_1"

--- ТЕСТ 2: ОЧЕРЕДЬ (QUEUE - FIFO) ---
[Очередь] Добавлен в хвост (enqueue): "Клиент_А".
[Очередь] Добавлен в хвост (enqueue): "Клиент_Б".
[Очередь] Добавлен в хвост (enqueue): "Клиент_В".
[Очередь] Содержимое (от головы к хвосту):
  1) "Клиент_А" <-- Голова (HEAD)
  2) "Клиент_Б"
  3) "Клиент_В" <-- Хвост (TAIL)
[Очередь] Обслужен из головы (dequeue): "Клиент_А".
[Очередь] Содержимое (от головы к хвосту):
  1) "Клиент_Б" <-- Голова (HEAD)
  2) "Клиент_В" <-- Хвост (TAIL)

--- ТЕСТ 3: ПРИОРИТЕТНАЯ ОЧЕРЕДЬ (PRIORITY QUEUE) ---
[Приоритетная очередь] Добавлен: "Фоновая_синхронизация" (Приоритет: 1).
[Приоритетная очередь] Добавлен: "Критический_сбой_системы" (Приоритет: 10).
[Приоритетная очередь] Добавлен: "Отправка_логов" (Приоритет: 2).
[Приоритетная очередь] Добавлен: "Срочный_запрос_пользователя" (Приоритет: 8).
[Приоритетная очередь] Добавлен: "Запрос_базы_данных_1" (Приоритет: 5).
[Приоритетная очередь] Добавлен: "Запрос_базы_данных_2" (Приоритет: 5).
[Приоритетная очередь] Содержимое (по убыванию приоритета):
  1) "Критический_сбой_системы" | Приоритет: 10 <-- НАИВЫСШИЙ ПРИОРИТЕТ
  2) "Срочный_запрос_пользователя" | Приоритет: 8
  3) "Запрос_базы_данных_1" | Приоритет: 5
  4) "Запрос_базы_данных_2" | Приоритет: 5
  5) "Отправка_логов" | Приоритет: 2
  6) "Фоновая_синхронизация" | Приоритет: 1

[Тестирование поиска элемента]:
[Приоритетная очередь] Найден элемент "Отправка_логов" на позиции 5 (Приоритет: 2).
[Приоритетная очередь] Элемент "Несуществующий_элемент" не найден!

[Тестирование удаления элемента]:
[Приоритетная очередь] Элемент "Отправка_логов" успешно удален.
[Приоритетная очередь] Содержимое (по убыванию приоритета):
  1) "Критический_сбой_системы" | Приоритет: 10 <-- НАИВЫСШИЙ ПРИОРИТЕТ
```

- 2) "Срочный_запрос_пользователя" | Приоритет: 8
- 3) "Запрос_базы_данных_1" | Приоритет: 5
- 4) "Запрос_базы_данных_2" | Приоритет: 5
- 5) "Фоновая_синхронизация" | Приоритет: 1

[Последовательное обслуживание очереди по приоритету]:

[Приоритетная очередь] Извлечен наивысший приоритет: "Критический_сбой_системы" (Приоритет: 10).

[Приоритетная очередь] Извлечен наивысший приоритет: "Срочный_запрос_пользователя" (Приоритет: 8).

[Приоритетная очередь] Извлечен наивысший приоритет: "Запрос_базы_данных_1" (Приоритет: 5).

[Приоритетная очередь] Извлечен наивысший приоритет: "Запрос_базы_данных_2" (Приоритет: 5).

[Приоритетная очередь] Извлечен наивысший приоритет: "Фоновая_синхронизация" (Приоритет: 1).

[Приоритетная очередь] Очередь пуста.

=====

ТЕСТИРОВАНИЕ УСПЕШНО ЗАВЕРШЕНО!

=====

Анализ результатов тестирования:

1. Корректность работы Стека: при добавлении документов 1, 2, 3 последним был добавлен «Документ_3». При выполнении операции pop() именно он был первым извлечен из вершины, что полностью подтверждает дисциплину LIFO.

2. Корректность работы Очереди: элементы «Клиент_А», «Клиент_Б», «Клиент_В» добавлялись в хвост очереди. При вызове dequeue() первым был обслужен «Клиент_А», первым вставший в очередь, что доказывает соблюдение принципа FIFO.

3. Корректность работы Приоритетной очереди: при добавлении элементов с различными приоритетами (от 1 до 10) структура автоматически выстроила элементы строго по убыванию приоритета. «Критический_сбой_системы» (приоритет 10) занял первое место в очереди, опередив ранее добавленные элементы с меньшим приоритетом.

4. Сохранение очередности при равных приоритетах: элементы «Запрос_базы_данных_1» и «Запрос_базы_данных_2» имеют одинаковый приоритет 5. В очереди «Запрос_базы_данных_1» остался перед «Запросом_базы_данных_2», сохранив хронологический порядок добавления.

5. Поиск и удаление: элемент «Отправка_логов» был успешно найден по наименованию и корректно удален из середины связного списка с восстановлением целостности указателей.

7. Выводы

- Изучены теоретические основы организации динамических структур данных на базе связанных списков и указателей в языке программирования Си.
- Освоены практические навыки динамического выделения (malloc) и своевременного освобождения (free) оперативной памяти, исключающие утечки памяти.
- На основе базового односвязного списка успешно реализованы классические структуры данных Стек (LIFO) и Очередь (FIFO).
- Реализована Приоритетная очередь с автоматической упорядоченной вставкой элементов по убыванию приоритета и сохранением хронологического порядка (FIFO) для элементов с одинаковым приоритетом.
- Реализованы функции поиска и удаления произвольных узлов списка по строковому наименованию, а также полное освобождение динамических структур при завершении программы.
- Проведено всестороннее тестирование разработанных алгоритмов в среде Microsoft Visual Studio, полностью подтвердившее корректность их функционирования.